SociaLink

AI-Powered Augmented Reality Social Networking Application

Project synopsis and build context. Prepared for the expo and to be given to Claude Code as project context.

> **How to use this document.** Section 1 is the original project description exactly as written by the developer. Sections 2 to 12 explain the decisions, architecture, data, API, build order, expo plan and design notes. The UI frames are not final and will change, so the design section keeps UI separate from logic.

1. Original project description (verbatim)

SociaLink – AI-Powered Augmented Reality Social Networking Application

Introduction

SociaLink is a mobile application that combines AI, facial recognition, computer vision, and AR-style overlays to enable users to discover the social and professional profiles of voluntarily registered people around them. Users create profiles with details such as their name, skills, LinkedIn, GitHub, Instagram, and portfolio, and enroll their face. The application converts the face into a facial embedding instead of storing the original image. When the camera detects a registered person, their profile is displayed as an interactive card near their face. It can recognize multiple registered users simultaneously and displays “Unknown” when no reliable match is found.

Tools & Technologies

- Flutter & Dart – Android mobile application and UI/UX
- Python & FastAPI – Backend and APIs
- InsightFace / ArcFace – Facial embeddings and recognition
- OpenCV – Computer vision and camera processing
- MediaPipe – Face detection/tracking
- Supabase (PostgreSQL) – User profiles and embedding database
- Git & GitHub – Version control and APK distribution
- Android Studio & VS Code – Development
- ChatGPT / GitHub Copilot – AI-assisted development

Note: the last tool listed (ChatGPT / GitHub Copilot) is for development help only. The running app uses no AI chatbot or paid AI service. See section 2.

2. Goals, constraints and decisions

Goal. Build SociaLink as a working Android demo and present it at a college expo. A visitor should be able to enroll in under a minute and be recognized right after, with their profile card appearing next to their face in the camera view.

Constraints and decisions made so far:

- **Completely free resources.** No paid plans, no paid APIs, no credit card needed anywhere.
- **AI is used to build the project, not inside the running app.** Claude Code is used during development. After deployment there are no AI chat or cloud AI services. The only machine-learning

part is the open-source InsightFace face model running on the developer's own machine (and a small on-device face detector on the phone). This is a local model, not a paid API.

- **Platform:** Android only (APK shared through GitHub Releases).
- **Opt-in only.** Only people who register and give consent can be recognized. Everyone else shows “Unknown” and nothing about them is stored.
- **No face photos stored.** Only numeric face embeddings are saved.
- **Developer level:** early-intermediate. Code should be simple, well commented and built in small steps that can each be tested.
- **UI frames are not final.** Design inspiration is still being collected and the look will change, so keep UI code separate from logic.

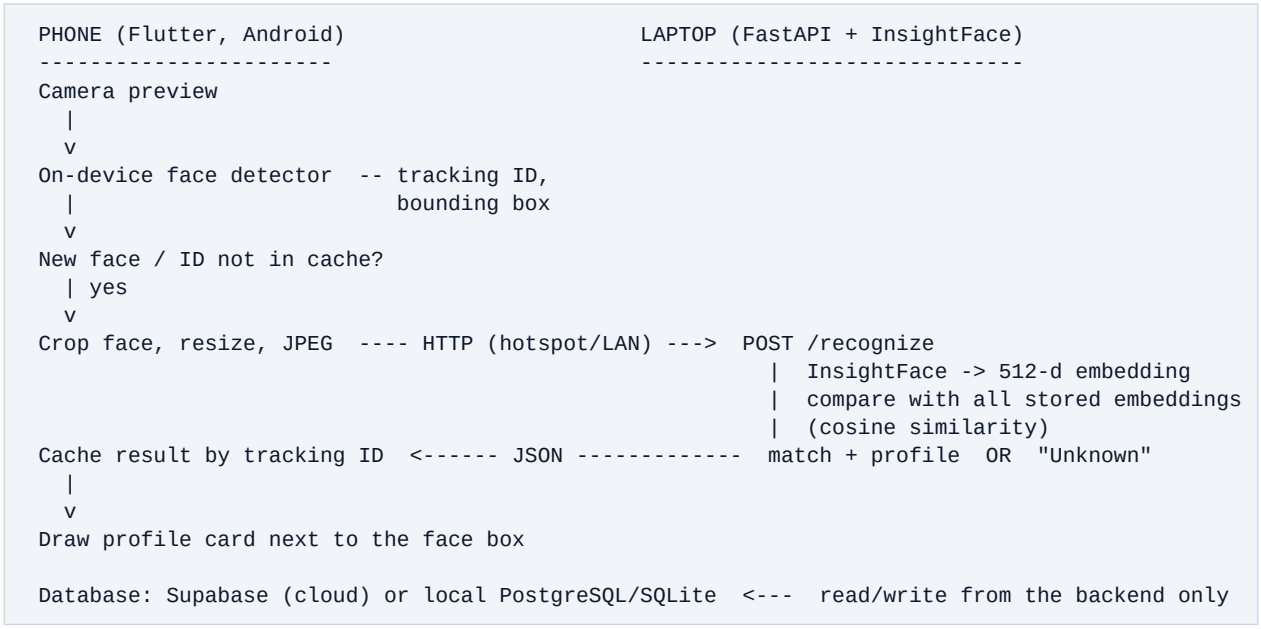
3. Free technology stack

Everything below is free for this use. The “catch” column lists the limits to remember.

Layer	Tool	Role	Catch / note
Mobile app	Flutter + Dart	Android UI, camera, overlays	Free. Test on a real phone, not only the emulator.
Backend	Python + FastAPI	Enroll, recognize, delete APIs	Free. Run on the laptop for the expo.
Face model	InsightFace (ArcFace)	Face detection and 512-number embeddings	Code is open-source, but the pretrained models are for non-commercial use. Fine for a college expo.
Computer vision	OpenCV	Image decoding, cropping, resizing	Free.
Face detection on phone	Google ML Kit face detection (or MediaPipe)	Draws a smooth box that follows each face	Free and on-device. See section 4 for why this is suggested.
Database	Supabase (PostgreSQL)	Profiles, embeddings, consent log	Free tier is about 500 MB. Projects pause after about a week of inactivity, so wake it up before the expo.
Offline alternative	Local PostgreSQL or SQLite	Same data on the laptop	Use this if there is no internet at the expo.
Version control	Git + GitHub	Code and APK releases	Free. Upload the APK as a GitHub Release.
Editors	Android Studio, VS Code	Development	Free.
Dev assistant	Claude Code	Builds the project	Development only, not part of the running app.

4. Architecture

The recommended design is a hybrid. The phone does the quick work (finding and tracking faces so the overlay follows smoothly). The laptop backend does the heavy work (turning a face crop into an embedding and matching it). Results are cached per tracked face, so the backend is only called when a new face appears.



Why this design

- Sending whole video frames to a server every frame makes the overlay laggy. A local detector keeps the box glued to the face at camera speed.
- Only small face crops go over the network, and only when a face is new or its cached result has expired.
- The backend runs on the laptop and the phone connects over the laptop's hotspot. This avoids dependence on unreliable expo Wi-Fi and avoids cold starts on free hosting.
- The phone never talks to the database directly. This keeps the Supabase keys off the phone.

Simpler fallback (if the hybrid is too much at first): the phone sends a resized full frame about 3 times per second, the backend detects, embeds and matches, and returns boxes and profiles. Slower and laggier, but fewer moving parts. Build this first if needed, then upgrade.

5. Recognition pipeline

Enrollment

- User fills in the profile and accepts the consent screen.
- App captures 3 to 5 face crops (front, slightly left, slightly right) and sends them to `/enroll`.
- Backend creates an embedding for each crop, normalizes it to length 1, and stores them. The photos are discarded immediately and never saved.
- Optional liveness step: ask the user to turn their head or blink on camera, so a printed photo of someone else cannot enroll them.

Recognition

- Backend loads all stored embeddings into one in-memory NumPy matrix when it starts.
- For a new face, compute its embedding and take the dot product with the matrix (cosine similarity, because all vectors are normalized).
- If the best score is at least the threshold, return that person's profile. Otherwise return “Unknown”.

- Start with a threshold around 0.45, then tune it by testing real pairs of same-person and different-person photos. Record the final value and how it was chosen, because judges may ask.
- With fewer than about 100 users, plain NumPy is fast enough. No vector database is needed.

Making it feel fast

- Cache by tracking ID for around 5 to 10 seconds so the same face is not re-sent every frame.
- Re-check a face every few seconds in case the first answer was wrong.
- Use the smaller InsightFace model (buffalo_s) if the laptop CPU is slow.
- Handle several faces at once: each tracked face has its own cache entry and its own card.

6. Data model

Three tables are enough. Embeddings are stored as arrays of 512 numbers (or as a pgvector column if using Supabase).

```
users
  id              uuid  primary key
  name            text
  headline        text          -- e.g. "Flutter developer"
  skills          text[]
  linkedin_url    text
  github_url      text
  instagram_url   text
  portfolio_url   text
  visible         boolean       -- user can switch recognition off
  created_at      timestamptz

face_embeddings
  id              uuid  primary key
  user_id         uuid  references users(id) on delete cascade
  embedding       float4[512]   -- or vector(512) with pgvector
  created_at      timestamptz

consents
  id              uuid  primary key
  user_id         uuid  references users(id) on delete cascade
  consent_version text
  accepted_at     timestamptz
```

Deleting a user removes their profile, embeddings and consent record together (cascade). No original face image exists anywhere to delete.

7. Backend API (FastAPI)

Endpoint	Purpose	Input	Output
POST /enroll	Create a user and store face embeddings	Profile fields, consent flag, 3 to 5 face crops	user id
POST /recognize	Identify faces in a crop or frame	One face crop, or a resized frame	List of {box, name, profile card data, score} or "Unknown"
GET /profile/{id}	Fetch a full profile	user id	Profile fields
PATCH /profile/{id}	Edit profile or toggle visibility	Fields to change	Updated profile
DELETE /profile/{id}	Delete my data	user id + proof it is the owner	Confirmation

Endpoint	Purpose	Input	Output
GET /health	Check the server is up	None	ok

Security basics for a demo

- Keep the Supabase service key only in the backend's environment file. Never put it in the Flutter app or on GitHub.
- Add a simple shared API key header between the app and the backend.
- Add `.env` to `.gitignore`.
- The delete endpoint must check that the caller owns the profile (for example with a secret token issued at enrollment).

8. Flutter app: screens and modules

Screen	What it does
Splash / welcome	Name, one-line explanation, "Get started".
Consent	Plain-language explanation: what is stored (embedding only), who can see the profile, how to delete. Must be accepted to continue.
Profile form	Name, headline, skills, LinkedIn, GitHub, Instagram, portfolio.
Face enrollment	Camera with guide oval, capture front/left/right, optional liveness prompt, progress indicator.
AR camera (main screen)	Live camera, face boxes, profile cards next to faces, "Unknown" label, connection status indicator.
Profile card (expanded)	Tap a card to see full profile with tappable links that open LinkedIn, GitHub, Instagram, portfolio.
My profile / settings	Edit profile, visibility switch, delete my data button.

Suggested folder structure

```
socialink/
  backend/
    app/main.py            FastAPI app and routes
    app/face_engine.py     InsightFace load, embed, compare
    app/db.py              Supabase or local DB access
    app/models.py          request/response schemas
    requirements.txt
    .env.example
  app/                     Flutter project
    lib/core/              api client, config, theme tokens
    lib/features/enroll/
    lib/features/camera_ar/
    lib/features/profile/
    lib/widgets/           profile_card.dart, face_box.dart (UI only)
  docs/                    this synopsis, design notes
  README.md
```

9. Privacy, consent and ethics

A face embedding is still biometric data, so this part must be solid. Judges are very likely to ask about it.

- **Opt-in only:** a person is recognizable only after registering and accepting the consent screen.

- **No photos stored:** only embeddings. Embeddings cannot be turned back into a clear photo.
- **Control:** the user can hide themselves (visibility switch) or delete everything at any time.
- **Unknown means nothing is stored:** faces that do not match are not saved, logged or sent anywhere else.
- **Misuse:** someone enrolling another person is the main risk. Mitigate with a live selfie capture at signup and a liveness prompt, so you can only enroll the face in front of the camera.
- **Law:** be ready to mention India's Digital Personal Data Protection Act, 2023, which covers consent and the right to erasure for personal data. This is awareness for the expo, not legal advice.
- **Accuracy and bias:** state honestly that results vary with lighting, masks, glasses and similar-looking people, and that the threshold was chosen to prefer showing “Unknown” over a wrong match.

10. Build order (milestones)

Each milestone should work and be tested before the next one starts.

#	Milestone	Done when
1	Face matching script in Python	Two photos of the same person score high, different people score low. Threshold chosen.
2	FastAPI backend with /enroll and /recognize (local DB first)	Can enroll from test images and recognize a new image using curl or the Swagger page.
3	Connect Supabase (or keep local DB)	Profiles and embeddings persist across restarts.
4	Flutter camera screen	Live camera works on a real phone and shows face boxes.
5	Recognition from the app	Phone sends a crop over the hotspot and gets a name or Unknown.
6	Profile card overlay	Card follows the face, supports 2 or more faces at once.
7	Enrollment flow with consent	A new person can sign up in the app and be recognized right away.
8	Delete, hide, edit profile	Delete removes all data from the database.
9	Polish and new design frames	Final UI applied, no logic changes needed.
10	Expo rehearsal	Full demo run twice in expo-like conditions, backup video recorded.

11. Expo plan

60-second demo flow

- One-line problem: "At events, you never know who someone is or what they do."
- Enroll a visitor live, including the consent screen.
- Point the camera at them and show their card appear.
- Point at someone not enrolled and show "Unknown" with nothing stored.
- Show the delete-my-data button.

Risks and fixes

Risk	Fix
Bad expo Wi-Fi	Run the backend on the laptop and connect the phone through the laptop hotspot. Use a local database for a fully offline demo.
Lag	On-device detection and tracking, send crops only, cache results by tracking ID.
Poor lighting or crowded background	Test in a bright spot. Keep the phone steady. Have a backup screen recording of a perfect run.
Few people enrolled	Pre-enroll 8 to 10 volunteers (classmates, teammates) so the demo always works.
Supabase paused	Open the project a day before and confirm it is active, or switch to the local database.
Phone battery / laptop sleeping	Bring chargers. Turn off sleep on the laptop. Keep a second phone with the APK installed.
Likely judge questions	Privacy, misuse, accuracy, why not just LinkedIn. Prepare a short answer to each (see section 9).

12. Design notes (frames will change)

The visual design is still being collected, so the code should make restyling easy:

- Put colors, fonts, spacing and corner radius in one theme file (design tokens). Changing the look should mean editing this file.
- Keep UI widgets (profile card, face box, buttons) in a `widgets` folder with no network or database code inside them.
- The profile card takes a simple data object (name, headline, skills, links) so any new card design can be dropped in.
- On the AR screen, prefer a semi-transparent card so it does not hide the face, and place it beside or above the face box.
- Use one consistent style (dark and futuristic, or clean and light) across all screens.

Search terms for design inspiration

Screen	Search for
AR camera / overlay	AR app UI, camera overlay UI, face scan UI, augmented reality HUD
Profile card	profile card UI, digital business card app, glassmorphism profile card
Whole app	social networking app UI, event networking app, professional networking app
Signup and consent	face enrollment UI, biometric consent screen, onboarding permissions screen
Profile and settings	edit profile UI, social links profile page, delete account screen

Where to look: Figma Community, Dribbble, Behance, Pinterest, Mobbin and Refero. Use them for inspiration only and do not copy a design exactly.

13. Instructions for Claude Code

When using this document as context, follow these rules:

- Use only free tools and libraries. Do not add paid services, paid APIs or anything that needs a credit card.
- Do not add any AI chatbot, LLM or cloud AI API to the running app. Face recognition must use the local open-source InsightFace model.
- Never store face photos. Store embeddings only. Discard images right after processing.
- Never put secret keys in the Flutter app or commit them to Git. Use a `.env` file and provide `.env.example`.
- The developer is early-intermediate. Write simple, commented code, explain what each new file does, and work one milestone at a time (section 10).
- Keep UI code separate from logic and use a single theme file, because the design frames will change.
- Always include consent, hide and delete-my-data features. They are required, not optional.
- Make the backend runnable on a laptop with one command, and make the app's server address easy to change in one config file.
- Before moving to the next milestone, tell the developer how to test the current one.

Open questions to confirm with the developer before coding

- Use Google ML Kit for on-device face detection in Flutter, or keep to MediaPipe as originally planned? (ML Kit is easier in Flutter.)
- Supabase in the cloud, or a local database on the laptop for the expo?
- Include the liveness step (blink or head turn) at enrollment, or skip it for the first version?
- Which UI style (dark futuristic or clean light) will the final frames use?